MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Control Flow

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- Understanding how programs are executed
- RISC-V Instructions controlling execution

How does the processor know what to execute next?

- Program: just a list of instructions
  - How do we know which one to execute?
- Von Neumann model
  - Program is another form of data
  - A sequence of numbers in memory
  - Each instruction has an address
- Program Counter (PC)
  - A special register
  - Holds the address of the next instruction
  - E.g. PC = 10 → next instruction is `002080B3`$_{16}$

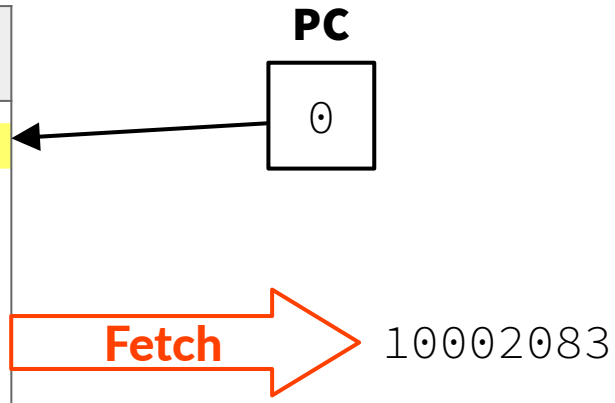
Address	Object Code	Mnemonic form
0	10002083	lw x1, a
4	10402103	lw x2, b
8	002080B3	add x1, x1, x2
C	10802103	lw x2, c
10	002080B3	add x1, x1, x2
14	10102623	sw x1, sum

The Fetch-Execute Cycle

- Initialisation
  - PC contains the address of the first instruction (normally 0)
- Fetch
  - Read the word in the location pointed to by PC
  - Increment PC by 4 (point it to the next memory word/instruction)
- Execute
  - Decode the instruction
  - Apply the operation on the operands
- Repeat forever*

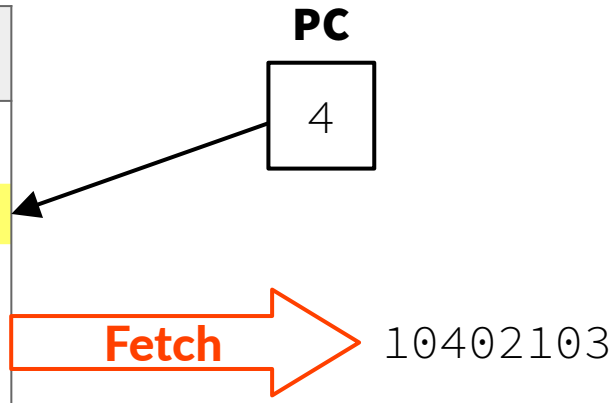
The Fetch-Execute Cycle

Address	Object Code	Mnemonic form
0	10002083	lw x1, a
4	10402103	lw x2, b
8	002080B3	add x1, x1, x2
C	10802103	lw x2, c
10	002080B3	add x1, x1, x2
14	10102623	sw x1, sum



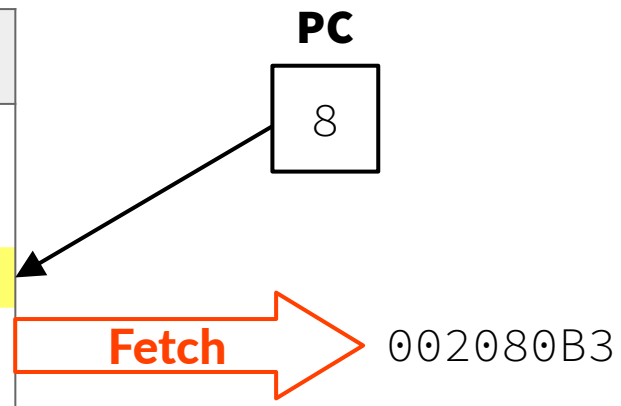
The Fetch-Execute Cycle

Address	Object Code	Mnemonic form
0	10002083	lw x1, a
4	10402103	lw x2, b
8	002080B3	add x1, x1, x2
C	10802103	lw x2, c
10	002080B3	add x1, x1, x2
14	10102623	sw x1, sum



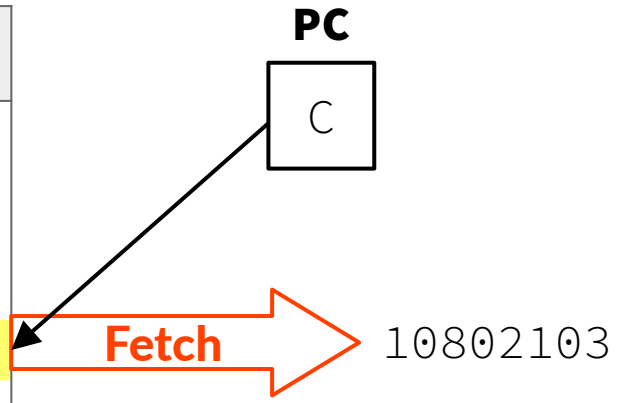
The Fetch-Execute Cycle

Address	Object Code	Mnemonic form
0	10002083	lw x1, a
4	10402103	lw x2, b
8	002080B3	add x1, x1, x2
C	10802103	lw x2, c
10	002080B3	add x1, x1, x2
14	10102623	sw x1, sum



The Fetch-Execute Cycle

Address	Object Code	Mnemonic form
0	10002083	lw x1, a
4	10402103	lw x2, b
8	002080B3	add x1, x1, x2
C	10802103	lw x2, c
10	002080B3	add x1, x1, x2
14	10102623	sw x1, sum



The Fetch-Execute Cycle

- Execution is serial
  - Going down the list of instructions
  - PC is incremented by 4 each time
- Little flexibility
  - Cannot repeat instructions without copy pasting code
  - Cannot change what is executed
  - Every time the program runs, it will apply the exact same operations
  - This is not how programs work!

RISC-V Control Flow Instructions

- Special instructions that arbitrarily change which instruction to execute next
  - They *effectively assign a value to PC*
  - Also known as *jump* or *branch* instructions
- Unconditional instructions
  - Always change the PC
- Conditional instructions
  - Change the PC only if a given condition is true

Unconditional Control Flow

- `jal rd, lab`
  - Jump and Link
  - Next instruction address stored in rd
    - rd <= PC + 4
  - Execution jumps to the address represented by label
    - PC <= label
- `jarl rd, rs1, imm`
  - Jump and Link with register
  - Next instruction address stored in rd
    - rd <= PC + 4
  - Execution jumps to the address represented by rs1 + imm
    - PC <= rs1 + imm

Unconditional Control Flow

- Jump and Link
  - Jump to a different instruction
  - And know how to go back to where you were
  - Powerful but too general (for now)
- Simple jump pseudo-instructions:
- `j label` → Jump to the label
  - Same as jal x0, label
- `jr rs1` → Jump to the address in the register
  - Same as jarl x0, rs1, 0

Writing to x0 has no effect

Conditional Control Flow

- General Syntax:
  - `b{cond} rs1, rs2, label`
  - Branch if cond
  - If cond is true for rs1 and rs2
    - PC <= label
  - If cond is false for rs1 and rs2
    - PC <= PC + 4

Conditions		
eq	rs1 = rs2	equal
ne	rs1 ≠ rs2	not equal
lt{u}	rs1 < rs2	less than
le{u}	rs1 ≤ rs2	less or equal
gt{u}	rs1 > rs2	greater than
ge{u}	rs1 ≥ rs2	greater or equal

Pseudo-instructions

Optional u indicates unsigned

Branch pseudo-instructions

- RISC-V ISA
  - The smallest instruction set that doesn't affect performance
  - If something can be efficiently done in the assembler or compiler, do it there
- blt & bgt are symmetrical
  - `blt x1, x2, target` is the same as
  - `bgt x2, x1, target`
  - `x1< x2 <=> x2 > x1`
- Same for ble & bge
- *ble{u} and bgt{u} implemented as pseudo-instructions*

More branch pseudo-instructions

- Common conditions
  - Is something zero or non-zero
  - Is something positive or negative?

And that's why we have x0

Zero Comparison Pseudo Instructions b<cond> rs, target		
beqz	rs = 0	beq rs, x0, target
bnez	rs ≠ 0	bne rs, x0, target
bltz	rs < 0	blt rs, x0, target
blez	rs ≤ 0	bge x0, rs, target
bgtz	rs > 0	blt x0, rs, target
bgez	rs ≥ 0	bge rs, x0, target

Conditions Example

- `beq x1, x2, label`
  - Jump to label if x1 = x2
- `bnez x1, label`
  - Jump to label if x1 is not 0
- `blt x1, x2, label`
  - Jump to label if x1 < x2
- `bgez x1, label`
  - Jump to label if x1 >= 0

Control Flow

- Necessary for
  - If-then-else statements
  - Loops
  - Functions/Methods
  - Complex logic operations
  - Runtime checks (e.g. overflow)
  - Will show how in future lectures

Summary

- Main execution loop
  - Fetch instruction
  - Execute instruction
- Program Counter (PC)
  - Indicates where the next instruction is
- Unconditional jumps
  - Go to a different part of the code
  - `jal` and `jarl`, `j` and `jr`
- Conditional branches
  - Decide what to execute next
  - Compare two registers
  - Jump if the condition is true
  - `beq, bneq, blt, bge`

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Representing information

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

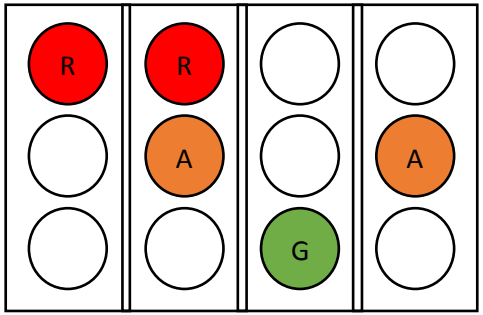
Overview

- How to represent real world information?
- How many bits do we need?
- What should these bits mean?

How to *think* about computers?

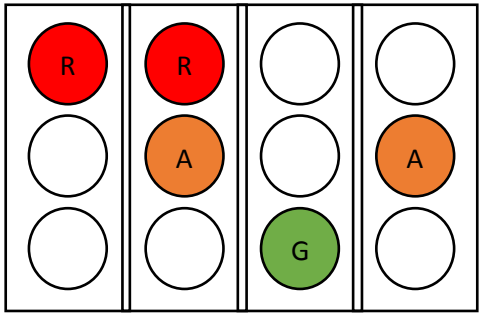
- Multiple levels of abstraction
  - Physics → Electrons and holes flow through semiconductors
  - Electronics → Voltages, transistors, memory cells
  - Computational → 0s and 1s, Binary numbers, Operators
- *All of them are equally valid!*
- At a basic computational level, all information is composed of bits: 0s and 1s
  - All values are can be represented as binary numbers
- Algorithms handle all different kinds of information
  - Not just binary
- How do binary numbers map to higher levels of abstraction?

What colour is the traffic light?



- Four possible values
  - "Red", "Red and Amber", "Green", "Amber"
- How can a computer represent colours?
  - *We work with numbers, not colours!*
    - No inherent correspondence from colour to number
    - ~~"Green definitely looks like a 53"~~
  - A fundamentally arbitrary choice
    - with some thinking involved

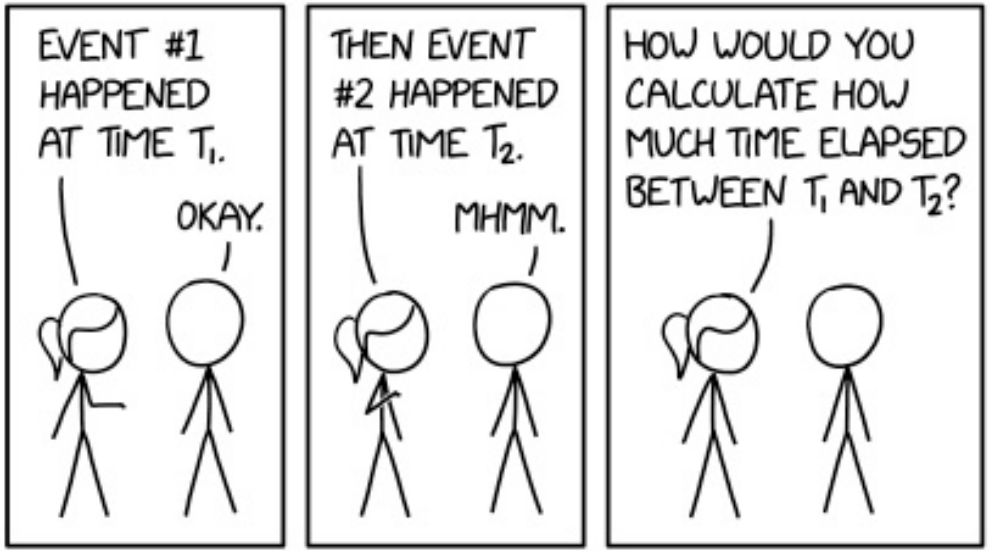
What colour is the traffic light?

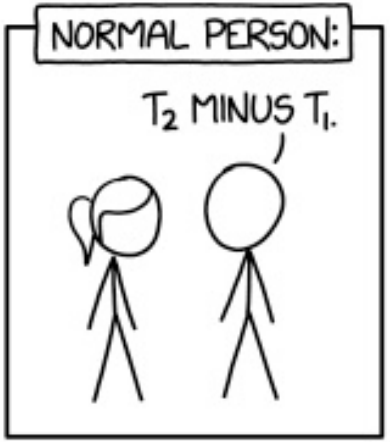


- Four possible values
  - "Red", "Red and Amber", "Green", "Amber"
- n bits can count 2^n things
  - *2 bits can count 4 different things*
  - 00, 01, 10, 11
- Still, 4! = 24 ways to map 4 values into 4 colours
- One good way
  - 00 <=> Green
  - 01 <=> Amber
  - 10 <=> Red
  - 11 <=> Red and Amber
- Another good way
  - 01 <=> Green
  - 00 <=> Amber
  - 10 <=> Red
  - 11 <=> Red and Amber

Can you think what makes these representations good?

What's the time?





© xkcd, from https://xkcd.com/2867/ license: CC BY-NC 2.5

What day of the week is it?

- 7 values
  - Monday, Tuesday, … Sunday
- 3 bits can count $2^3 = 8$ things
- Which seven numbers to use?
- Which number is which day?
  - Real world problem: Day 1 can be Monday, Saturday, or Sunday
  - 001 <=> Monday
  - 010 <=> Tuesday
  - 011 <=> Wednesday
  - 100 <=> Thursday
  - 101 <=> Friday
  - 110 <=> Saturday
  - 111 <=> Sunday
  - 000 is unused

System	First day of week	First week of year contains			Can be last week of previous year	Used by or in
ISO 8601	Monday	4 January	1st Thursday	4–7 days of year	yes	EU (exc. Portugal) and most of other European countries, most of Asia and Oceania
Middle Eastern	Saturday	1 January	1st Friday	1–7 days of year	yes	Much of the Middle East
Western traditional	Sunday	1 January	1st Saturday	1–7 days of year	yes	Canada, United States, Iceland, Portugal, Japan, Taiwan, Thailand, Hong Kong, Macau, Israel, Egypt, South Africa, the Philippines, and most of Latin America

What day of the month is it?

- Days 1-31 → 31 values
- 5 bits can represent 32 values
  - 1 <=> 00001, 2 <=> 00010…
- General approach
  - n bits can represent 2^n values
  - n bits can represent M values, if $M \leq 2^n$
  - n bits can represent M values, if $\log_2(M) \leq \log_2(2^n) = n$
  - *M values require $\log_2(M)$ bits (rounded up)*

And so on

- Student registration numbers used to be
  - 3 decimal digits for year
  - 5 decimal digits to count students
- Maximum number of students: 10^5 = 100,000 (0-99,999)
- How many bits needed to count the same students
  - $\log_2(100{,}000) \approx 16.61$, so 17 bits needed
- How many bits for the human population?
  - $\log_2(8{,}200{,}000{,}000) \approx 32.93$, so 33 bits needed

Summary

- Representing information is a matter of interpretation
- N bits can represent 2^N distinct values
- Many ways to map 2^N values to N bits
- Some choices arbitrary, some educated

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Character Representation

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Characters

- This is bit more important
  - A-Z → 26, a-z → 26, 0-9 → 10
  - 17 symbols, +2 for space and newline
  - Over 81 characters
- $\log_2(81) \approx 6.34$
- 7 bits (that can represent $2^7=128$) are enough

ASCII

- An early 7-bit standard – 128 English and American characters
- "American Standard Code for Information Interchange"
  - `A is represented by 100 0001`
  - `B is represented by 100 0010`
  - `Z is represented by 101 1010`
  - `a is represented by 110 0001`
  - `b is represented by 110 0010`
  - `z is represented by 111 1010`
  - `0 is represented by 011 0000`
  - `9 is represented by 011 1001`
  - `+ is represented by 010 1011`
  - `. is represented by 010 1110`

32

32

- Plus various ASCII codes for "non-printing characters" or "control codes"
  - `CR (Carriage Return, go to the start of the line) 000 1101`
  - `LF (Line Feed, go to the next line) 000 1010`
  - `FF (Form Feed, go to a new page) 000 1100`

What about us non-americans?

- Greek letters: α, β, γ, δ, ετς
- Latin alphabet variations: á, â, è, č, ű, ň, ë, ç, etc
- Cyrillic, Arabic, Chinese, Japanese
- ISO 8859 family for Alphabets + Abjads
  - 8 bits → 2^8=256 values
  - 128 for ASCII + 128 for extra characters for a specific script

Chaos!

- Code E4 → 1110 0100 maps to
  - Latin-based encodings → ä
  - ISO-8859-5 → ф
  - ISO-8859-6 → ل
  - ISO-8859-7 → δ
  - ISO-8859-8 → ה
  - ISO-8859-11 → ไ

What about us non-americans?

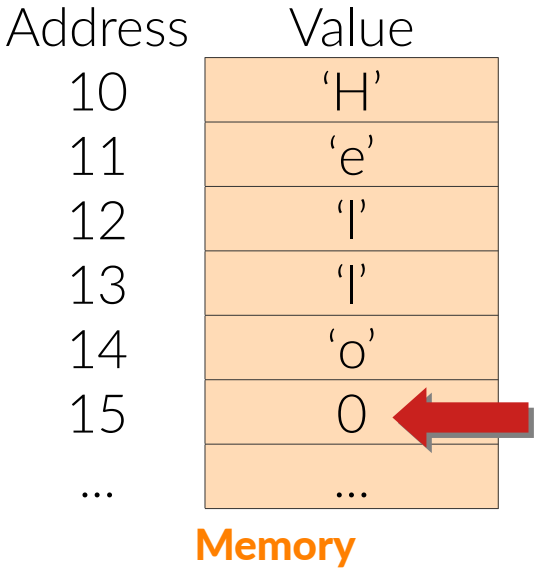
- UTF-8 (Unicode/ISO10646)
  - A superset of ASCII
  - If the 8th bit is 0, it's ASCII
  - If the 8th bit is 1, then encoding uses the next byte too (16 bits → 65,536 characters)
  - Can use up to four bytes (32 bits → More than 4 billion characters)
  - Most of internet, text files, code, increasingly internal representations
- The assembler translates characters to ASCII
  - `addi x1, x1, 'a'` translates to `addi x1, x1, 0x61`
  - 99% of the time you don't have to worry about this

Strings

- Words, sentences, paragraphs, etc
- Some sequence of characters
  - Any kind of sequence
  - Any kind of encoding
- A method for identifying the end
  - Multiple methods

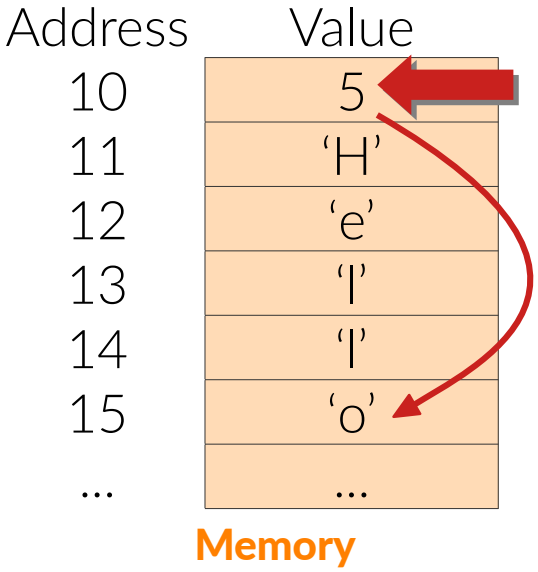
Strings

- In practice
  - Sequence means consecutive memory locations
  - Encoding means one of the standard encodings
  - Special end character or length attribute



Strings

- In practice
  - Sequence means consecutive memory locations
  - Encoding means one of the standard encodings
  - Special end character or length attribute



Strings

- Most common low-level approach
  - ASCII
  - 1 byte per character
  - Null terminating character (0)

Summary

- ASCII → Simply and compact but only for English
- UTF-8 → All possible languages but more complex
- Strings → Sequences of character in memory
- We use ASCII with simple null terminating strings

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Number Representation

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- How to represent integers?
- Main size options?
- What about negative numbers?

Integers

- Relatively easy: sets of bits → natively treated as base-2 numbers
  - Though, different representations might make sense (see Gray code)
- How many bits?
  - More bits → *better range*
  - Fewer bits → *less storage*

```
8 bits  →   0 to                     255
16 bits →   0 to                  65,355
24 bits →   0 to              16,777,215
32 bits →   0 to           4,294,967,295
64 bits →   0 to 18,446,744,073,709,551,615
```

- Python: the largest native type (doesn't care about storage)
- RISC-V: 8-bits (byte), 16-bits (half-word), 32-bits (word)
- 64-bit architectures: also double-word (64-bits)

Positional Significance (recap)

- Base-k
  - If we have N digits
  - They are numbered from 0 to N-1
  - Can count k^N numbers, 0 to (k^N-1)
  - Digit m represents k^m
- Base-10
  - Can count 10^N numbers, e.g. 4 digits → 0 to 9,999 → 10^4 = 10000 numbers
  - Digit m represents 10^m
  - $123_{10} = 1 * 10^2 + 2 * 10^1 + 3 * 10^0$
- Base-2
  - Can count 2^N numbers, e.g. 4 digits → 0 to 1111_2 (15_{10}) → 2^4 = 16 numbers
  - Digit m represents 2^m
  - $11010_2 = 1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 = 16 + 8 + 2 = 26$

Decimal → Binary

- $b_3b_2b_1b_0 = b_3 * 2^3 + b_2 * 2^2 + b_1 * 2^1 + b_0 * 2^0$
  - b_0 represents whether the number has 1
  - b_1 represents whether the number has 2
  - b_2 represents whether the number has 4
- $b_3b_2b_1b_0 \div 2 =$
  - $(b_3 * 2^3 + b_2 * 2^2 + b_1 * 2^1 + b_0 * 2^0) \div 2$
  - $(b_3 * 2^2 + b_2 * 2^1 + b_1 * 2^0) = b_3b_2b_1$, Remainder = b_0
- Repeatedly dividing by two returns the next bit as the remainder
  - $b_3b_2b_1b_0 \div 2 = b_3b_2b_1$, Remainder = b_0
  - $b_3b_2b_1 \div 2 = b_3b_2$, Remainder b_1
  - $b_3b_2 \div 2 = b_3$, Remainder b_2
  - $b_3 \div 2 = 0$, Remainder b_3

Decimal → Binary

- 21_{10} to binary
  - `21 / 2 = 10 remainder 1 → 1`
  - `10 / 2 =  5 remainder 0 → 01`
  - `5 / 2 =  2 remainder 1 → 101`
  - `2 / 2 =  1 remainder 0 → 0101`
  - `1 / 2 =  0 remainder 1 → 10101`
- $10101 = 2^4 + 2^2 + 2^0 = 16 + 4 + 1 = 21$

Usually the assembler can do this automatically

Negative Numbers

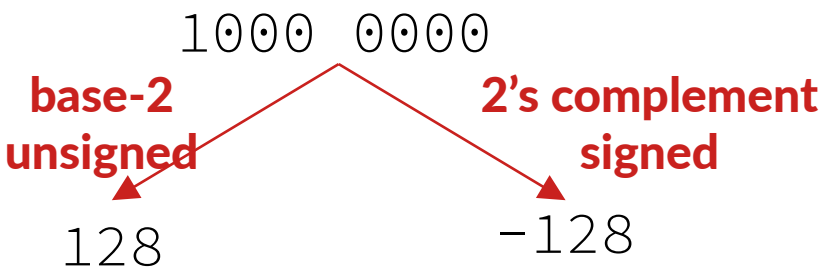
- Base-2 numbers are positive → We need another convention for negative numbers
- Most computers use *two's complement* (COMP12111) *but alternatives exist*
  - Positive numbers stay the same
  - Unique representation for each number
  - Addition remains straightforward
- Most significant bit = "*Sign Bit*"
- Range: (-2^{n-1}) to $(2^{n-1} - 1)$
  - 8 bits: -128 to 127
  - Still 2^n distinct values

Negative Numbers

- *Algebraic approach*: -x is represented as (2^N-x)
  - Representing -1 with 8 bits: 256 – 1 = 255 = $1111\ 1111_2$
  - Representing -128 with 8 bits: 256 – 128 = 128 = $1000\ 0000_2$
- *Alternative*: Lower N-1 bits represent the positive number, bit N-1 represents its negative value - (2^{N-1})
  - Representing -1 with 8 bits: -128 + 127 = $1\ 11111111_2$
  - Representing -128 with 8 bits: -128 + 0 = $1\ 0000000_2$
- *Binary logic*: -x is represented as the (inverse of x) + 1
  - Representing -1 with 8 bits: inverse($0000\ 0001_2$) + 1 = $1111\ 1110_2$ + $0000\ 0001_2$ = $1111\ 1111_2$
  - Representing -128 with 8 bits: inverse($1000\ 0000_2$) + 1 = $0111\ 1111_2$ + $0000\ 0001_2$ = $1000\ 000_2$

Negative Numbers

- Everything is about interpretation
- 2's complement useful because some operations work regardless of interpretation
  - addition, subtraction, equality comparisons
- Some others don't
  - We need to specify how to interpret them



Explicitly unsigned operations

- Less/greater comparisons & multiplication/division/remainder
- *Interpretation is needed*
  - bltu, bleu, bgtu, bgeu, sltu, sltiu, mulu, mulsu, divu, remu
- x1 <= FFFF $FFFF_{16}$ and x2 <= 0000 0002_{16}
  - x2 is always 2 (msb is zero)
  - x1 is $2^{32} - 1$ if unsigned, -1 if signed
- Comparison
  - If unsigned: x1 = 2^{32}-1 & x2 = 2 → x1 > x2
  - If signed: x1 = -1 & x2 = 2 → x1 < x2
- Division
  - If unsigned: x1 / x2 → $2^{31} - 1$ → 0FFF $FFFF_{16}$
  - If signed: x1 / x2 → 0 → 0000 0000_{16}

Extending numbers

- $89_{10} = 0089_{10}$
- $0111_2 = 0000\ 0111_2$
- $1000_2 = 0000\ 1000_2$ ***Only true if base-2!***
- In two's complement
  - $1000_2 = -8$
  - $0000\ 1000_2 = 8$
  - -8 in 8 bits → 256 – 8 → 248 → 128 + 64 + 32 + 16 + 8
  - $1111\ 1000_2 = -8$
- $1000_2 = 1111\ 1000_2 = 1111\ 1111\ 1111\ 1000_2$

Extending numbers

- For unsigned numbers → add zeros
- For signed numbers
  - Add 0s if positive (the sign bit is 0)
  - Add 1s if negative (the sign bit 1)
  - *Sign Extension* → Copy sign bit in front of number
- Some operations extend numbers
  - Signed/unsigned or arithmetic/logical instruction variants

Other number representations

- Other ways to "encode" information into binary, known as "formats" or "codes"
  - Base-2 integers (signed and unsigned) are just one common format
- Real numbers can be represented in this form:
  - E.g. $-1.1010_2 \times 2^{27}$ has sign (-), mantissa (1010), exponent (27)
  - In a computer, real numbers (or "float" in Python) can be represented in the same manner:

bit 31: *sign bit*	bits 30-23: *exponent*	bits 22-0: *mantissa*

- In IEEE 754 standard, 32-bit floating point format numbers are stored with at least 6 decimal digit precision and the largest value ≈ 3.402823×10^{38}
- https://en.wikipedia.org/wiki/Single-precision_floating-point_format
- COMP15111 focuses on Integer arithmetic

Summary

- Unsigned (base-2)
- Signed (2's complement)
- Explicitly unsigned instructions
- Floating point numbers